Wrench-Limited Block Extraction on the myCobot Pro 630

A suction pick-and-place policy that lifts a block from a tightly packed array under an admissible-wrench seal constraint, trained in a batched MuJoCo-Warp twin and distilled from a privileged scripted expert.

Repository: `frankkimrobotics/pick_n_place` · Code: `rl/wrench_extract.py`, `rl/wrench_pick.py`, `rl/real_wrench_ctrl.py` · Policy: `~/pnp_rl/wrench_extract/policy_final_12of12.pt`

---

1. Problem setup

1.1 Task

A 3×3 array of rectangular blocks is packed on a table with 2–5 mm gaps between neighbours. The target is the **centre block**. A robot arm carrying a vacuum suction cup must:

1. **Approach** from its home pose and **press** down onto the centre block until the cup seals;
2. **Extract** the block vertically far enough to clear its (taller, possibly) neighbours;
3. **Carry** it across the table to a drop box at a fixed location;
4. **Release** it inside the box.

The difficulty is twofold:

- **Unknown heights.** The block heights are drawn per scene from 30–60 mm and the policy is given only a *noisy belief* of them (σ = 6 mm), never the truth. It does not know how far it must lift before the block clears the pack — pull too little and the block snags a taller neighbour; the extraction must be robust to that uncertainty.
- **A force-limited seal.** The cup can only transmit a bounded wrench before it peels off. Lifting too hard, or swinging the block during the carry, drives the interface force outside the cup's **admissible wrench set**, and the block detaches. The safe operating window is narrow and changes with every scene's geometry.

1.2 Platform and simulation

The target hardware is a **myCobot Pro 630** (6-DOF) with a vacuum cup end-effector, driven through a LinuxCNC/ `robot_hal` stack on a Raspberry Pi. Training is done entirely in a **batched MuJoCo-Warp twin** (`env_warp.PickEnv`) that reproduces the arm's *measured* velocity-mode drive (saturation, acceleration cap, command dead-time, following-error limit, and the deployed streaming control law), so that behaviour transfers to the physical drive rather than to an idealized PD plant. Hundreds of worlds step in parallel on the GPU.

1.3 The wrench-set suction model

The seal is modelled (`wrench_pick._install_wrench_model`) as a compliant spring that pulls the held block toward its latched pose, with the **transmitted wrench clamped to an admissible set** in the cup frame. With vacuum force `F_V = 18 N`, friction coefficient `μ = 0.5`, and effective cup radius `R_eff = 8 mm`, writing `head = max(0, F_V − pull)`:

```
pull      ≤  F_V                        (tension the vacuum can hold)
|shear|   ≤  μ · head                   (friction, shrinks as you lift harder)
|peel|    ≤  0.6 · R_eff · head         (moment that pries the lip)
|torsion| ≤  (2/3) · μ · R_eff · head   (twist about the cup axis)
```

The key structural fact is **coupling**: as the lifting tension `pull` rises toward `F_V`, the head-room for shear, peel and torsion collapses toward zero. The transmitted wrench is projected into this set each physics substep; demanding beyond it for `BREAK_TICKS = 150` consecutive substeps (≈ 0.3 s, a real cup lip deforms before peeling) **detaches** the block. The last interface wrench and its margin are policy observations.

1.4 Observation and action spaces

- **Observation** — 33-D state vector: joint angles (6), joint velocities (6), **tool pose expressed relative to the array centre** (3), cup z-axis (3), object-relative offset (3), box-relative offset (2), height beliefs (target, neighbour-max, believed clearance) (3), wrench readout (pull, shear, peel, torsion, margin) (5), sealed flag (1), episode phase (1). Gaussian noise ($\sigma = 0.005$) is injected at train and test time. **This is a state vector, not an image** — on the real arm its entries come from the depth + segmentation perception stack.
- **Action** — 7-D: per-joint target deltas (6), clamped to 3.2°/decision (≈ 32°/s at 10 Hz, under the drive's ~36°/s following-error limit), plus a suction command logit (1).

1.5 Randomization

Per *variant* (MuJoCo-Warp geometry is model-level, so it varies across variants rather than across worlds): block half-width 15–26 mm, inter-block gap 2–5 mm, array centre $x \in [0.30, 0.40]$, $y \in [-0.06, 0.06]$. Per *world/episode*: block heights 30–60 mm with noisy beliefs, home-pose joint noise, and drive domain randomization (velocity/acceleration/dead-time multipliers).

---

2. Method

The final policy is produced by **behaviour cloning from a privileged scripted expert**. Reinforcement learning was used throughout as a *diagnostic* — it validated the reward and gating design and, decisively, exposed four simulator physics bugs — but it never produced the deployed weights. The method is best read as the sequence of problems solved.

2.1 Build the environment on the proven twin

`ExtractEnv(PickEnv)` subclasses the existing, numerically-stable batched pick environment rather than a from-scratch scene. Episodes can start in any of four modes (see §2.5). The first blocker was **silent NaNs**: a tight pack of nine free bodies overflows MuJoCo-Warp's default constraint buffers (`njmax = 64`), and dropped constraints corrupt the solve. `PickEnv` already exposes a `data_kw` hook for exactly this — setting `njmax = 512, nconmax = 256` eliminated it. (This same overflow had killed an earlier from-scratch version that was misdiagnosed as a timestep/stiffness problem.)

2.2 Find the physics bugs by *tracing the expert*

Seven early RL runs plateaued while we tuned rewards and hyperparameters. The breakthrough was to stop tuning and instead run a scripted expert through the environment and **decompose the interface force**. That exposed four

defects in the simulated contact — not in the learning — each verified by a force trace. Before the fixes, **0 %** of carries survived; after, the expert reached **128 / 128**.

#	BUG	SYMPTOM (TRACED)	FIX
1	**Inverted cup axis**	`n_ax = −R[:,:,2]` negated the cup's pointing vector; tension and compression were swapped, so true vacuum pull was never checked against `F_V`.	Use the tool z-axis directly (it already points at the block).
2	**Damper on absolute velocity**	The seal damper acted on world-frame object speed: a 0.5 m/s carry produced ~15 N of phantom drag-shear against an 8.6 N shear budget, so *every* transport broke the seal.	Damp the **relative** cup–object velocity (track the cup pose per substep).
3	**Double-modelled contact**	A rigid cup-tip ↔ block contact fought the suction spring once sealed — 9 N standing preload, 26 N spikes — manufacturing breaks no real cup would see.	`<contact><exclude>` the cup–block pair while sealed.
4	**Carry snagged the wall**	The block hangs ~5 cm below the tool and the box walls top out at 0.25 m; a low carry path dragged the block into the rim and sheared it off.	Carry at z = 0.33 m, above every obstacle.

2.3 The scripted waypoint expert

The expert (`teacher_action`) is a privileged, batched state machine over joint-space waypoints: **transit** (home → hover over the array) → **two-stage press** (fast to a pre-press point above the tallest possible block, then a slow straight descent so the geometric latch catches the block at its *true* height) → **extract** (vertical lift) → **carry** (via a mid waypoint to over-box, at safe altitude) → **release** (only when over the box). Two refinements mattered:

- **Direction-preserving tracking.** A naïve per-joint box clamp on the tracking action bows the tool path ~15 mm sideways and misses the 12 mm latch gate. Scaling all joints uniformly so the largest moves at the target rate keeps the path straight. (This is the same direction-distortion issue seen on the real robot.)
- **A margin governor.** The transmitted-wrench margin scales the commanded joint deltas down when the seal is loaded, making the extraction and carry survivable by construction — the expert's "wrench-modulated speed" made structural.

2.4 Suction-executor gating

Release is **physically gated**, mirroring how a real executor guards the pump: the cup only lets go when the block is *both* cleared and over the box. This closes a persistent reinforcement-learning exploit — a free release logit collapses episodes into a "give up instantly" optimum — and matches deployment. The seal latches geometrically (cup within gate distance of the block top at low relative velocity).

2.5 Curriculum

To make the long-horizon carry and place discoverable, episodes start in a mix of modes: **40 % home** (the full task, including approach and press), **20 % sealed in the array**, **15 % mid-carry**, **25 % over-box**. Value learned from the easy over-box starts propagates back to teach the carry, which pure exploration from home never found. Metrics track full-task (home-start) success separately so the headline number is honest.

2.6 Distillation recipe (behaviour cloning)

With the expert at 128/128, the policy is cloned from ~800k (state, teacher-action) pairs collected across 32 geometry variants under domain randomization. Three settings were each *necessary*:

- **Network width** `h = 512`. `h = 256` underfits the 32-variant geometry diversity (0/12).

- **Canonical observations.** Expressing the tool pose *relative to the array centre* rather than in absolute workspace coordinates. Absolute coordinates keyed the policy to each training layout — it was the generalization ceiling (9/12), and the cause of later PPO overfitting.
- **DART-style exploration noise above the action noise floor.** Rollout noise large enough (0.12 on the press phase, phase-scaled) that the expert *demonstrates its own corrections*. Too little noise teaches open-loop tracking and the clone drifts through the latch gate; press descent actions below ~0.25 of the clamp sink below the regression noise floor and never clone (round v13 had the best training loss of any round and transferred at 0/12).

2.7 What did not work

- **DAgger consistently regressed** (9/12 → 0/12, twice). The waypoint expert's label generator is *stateful* — its phase index advances on its own trajectory — so labels queried on the policy's off-trajectory states are systematically wrong. DAgger requires a stateless/trajectory-free expert; this one is not.
- **PPO fine-tuning did not improve the clone.** Warm-started from the 12/12 clone it either overfit (the earlier absolute-coordinate obs drove 9/12 → 4/12) or, with canonical obs, drifted out of the clone basin and collapsed at ~130k steps despite a KL cap; the score-gated best checkpoint came out identical to the clone.

2.8 Speed analysis

To ask "can it go faster without violating the constraint", the policy's joint-delta actions were amplified at inference by 1.0×, 1.3×, 1.6×. All three gave the **identical** result (12/12, 0 breaks, 106 decisions). Action saturation by phase confirms why: the policy uses only ~56 % of the action clamp in free motion and ~72 % when sealed — it is **not** speed-limited by timidity. The cycle is dominated by phases that are slow *by constraint* (the low-velocity press the latch requires, and the margin-governed extract/carry), where the margin *is* the safety buffer.

2.9 Real-robot deployment path

`real_wrench_ctrl.py` runs the policy through the controller already proven on this hardware: the `PiLink` streaming law (K0/K1/vff gains, lead-limited reference), the torque-contact guard (firm/hard thresholds), scripted approach/retract legs, and attach emulation. The 33-D observation maps directly onto the perception stack. Deployment bring-up was validated: MPC stack relaunched, 2 ms feedback, a 1° joint probe confirming the drives follow, and object localization from the calibrated fixed D405 depth camera. Safety is bounded by a joint-space guard box, torque abort, box-gated release, a touch-only demo mode, and an interrupt that stops the stream and drops vacuum.

---

3. Results

3.1 Headline

Evaluated from the **home pose** on 12 scenes with geometry never seen in training (random block size, gap, array position, and heights):

METRIC	VALUE	NOTE
Placed in box	**12 / 12**	unseen randomized geometry
Seal breaks	**0 / 12**	wrench margin stayed positive throughout
Neighbour disturbance	~28 mm	total shift summed over 8 neighbours
Cycle time	~10.3 s	home → approach → seal → extract → carry → place

METRIC	VALUE	NOTE
Training scale	512 worlds × 32 variants	

The scripted expert it is cloned from is **128 / 128** across all curriculum modes under domain randomization.

3.2 Training journey

Unseen-scene success across the cloning rounds (the dips are specific, understood imitation failures, not noise):

ROUND	CHANGE	RESULT	LESSON
v3	fixed geometry, home start	6/6	baseline works
v4	+ random geometry (16 var, h256)	3/6	absolute-coord obs overfit each layout
v5	+ DAgger	0/6	stateful teacher mislabels off-trajectory states
v6	32 var, clean demos (h256)	0/6	h256 underfits the diversity
v8	**h512 + DART noise**	**9/12**	bigger net; teacher demonstrates corrections
v9	DAgger on v8	0/12	DAgger regresses again — poison confirmed
v12	sharp alignment-gated teacher	4/12	a discontinuous teacher clones badly
v13	smooth, slow (0.15) descent	0/12	press actions below the clone's noise floor
v14	**canonical obs + 0.25 descent + runway**	**12/12**	array-frame obs + press above the noise floor

3.3 Cycle-time breakdown

PHASE	DECISIONS	TIME	RATE-LIMITED BY
transit + press (home → seal)	40	4.0 s	latch needs low approach velocity (physics)
extract (seal → cleared)	5	0.5 s	already minimal
carry + place (cleared → released)	58	5.8 s	waypoint path + vacuum dwell + settle

3.4 Speed test

ACTION GAIN	PLACED	BREAKS	DECISIONS
1.0×	12/12	0	106
1.3×	12/12	0	106
1.6×	12/12	0	106

Flat — free-motion speed is not the bottleneck.

3.5 Visual robustness

A showcase run (`wrench_showcase.py`) randomizes lighting, colour and block palette on top of the geometry randomization: **6 / 6, zero breaks**, behaviour unchanged. Because the policy is state-based, appearance is irrelevant to it by construction.

3.6 Hardware status

Bring-up verified (feedback stream, probe passed, camera localization, controller ready). A full physical pick was not completed in the session — object localization needs the final touch-confirmation step on the candidate the camera returned.

---

4. Discussion

Debug the physics before the policy. The single most important methodological lesson is that ~7 RL runs were spent tuning rewards and hyperparameters against blockers that were actually simulator contact bugs. Tracing a scripted expert through the same environment and decomposing the interface wrench found all four in under an hour. When a manipulation policy plateaus, instrument an analytic expert and look at the forces before touching the reward.

RL as instrument, not generator. On this task reinforcement learning was most valuable as a diagnostic: it proved the task was solvable, validated the reward shaping and the release gating, and surfaced the physics. But the PPO weights never beat a clone of the (now-correct) expert — warm-start fine-tuning either overfit or collapsed. The deployed artifact is a behaviour clone. This mirrors a prior finding on this platform's measured drive, where DAgger/imitation beat PPO.

DAgger needs a stateless expert. DAgger's premise is that the expert can label any state the learner visits. A waypoint state machine violates this: its action depends on a phase index tied to its own trajectory, so its labels on the learner's drift states are wrong in a systematic, confidence-preserving way — which is worse than noise. Every DAgger round here regressed. The fix is not more rounds; it is an expert that is a pure function of state (or re-deriving the phase from state at query time).

Behaviour cloning has two quiet prerequisites. First, **canonical/relative observations** — the clone generalizes across geometry only if the observation is expressed in a frame that factors out the thing being randomized; absolute coordinates silently memorize layouts. Second, the demo actions in the phases that matter must sit **above the regression noise floor** — a "safer", slower expert whose critical actions shrink toward zero produces a lower training loss and a worse policy, because the loss is dominated by the large transit actions and the small press corrections are averaged away.

The speed ceiling is the constraint, by design. The honest answer to "make it faster" is that the free-motion speed is already near the drive's limit, and the remaining cycle time lives in the seal-limited press and carry. The margins that look like slack (0.7–0.9) are the safety buffer that keeps breaks at zero; spending them is precisely the constraint the task exists to respect. Genuine speedup, if required, is a *carry-path* problem (a more direct trajectory, carefully regularized to avoid the drift collapse) or a tuning of the fixed vacuum-dwell/settle windows — not a global velocity increase.

4.1 Limitations and future work

- **Sim-to-real is validated at the interface but not yet closed on a full pick.** The next step is the touch-confirmed localization and a touch-only then full run on the physical arm.
- **The wrench model is a designed approximation** of a real vacuum cup; the `F_V`, `μ`, `R_eff`, and break-tick constants should be calibrated against the physical cup (sloped or porous tops do not seal, per prior hardware series).
- **A stateless teacher** (re-deriving the waypoint phase from state) would re-open DAgger and on-policy vision distillation.

- **A vision student**: because the state observation maps onto depth+segmentation outputs, the state policy can label on-policy RGB-D rollouts — proper DAgger with a stateless teacher — to train an image-conditioned policy.
- **The scripted-expert → clone stack is a strong data source** for the differentiable-MPC (DiffOG) line of work: batched policy rollouts give unlimited (observation, trajectory) pairs for the transformer-predicts-cost imitation objective.

Generated as part of the pick_n_place program. Interactive companion page: `docs/wrench_extract.html`.